

PLAN → target-lookup → chembl-actives → mol-properties → selectivity-check → REPORT

각 화살표마다 검증 게이트: verification.passed = false 면 다음 단계 진행 금지

PDE5 저해제 탐색 에이전트 하네스

— 강사 데모

자율 에이전트가 실제로 어떻게 도구를 쓰는가

재직자 3과정 · 6일차(9/5) · 대규모 언어모델과 신약개발 · 강사 박혜진

오늘 화면에서 볼 것 — 하네스 해부 → 도구 실행 → 원칙

1. 하네스 해부

폴더 구조 · CLAUDE.md 규약 · 스킴 6종 / 스크립트 7종 · JSON 출력 계약

2. 도구 5종 실행

target-lookup · chembl-actives · mol-properties · selectivity-check · verify (실제 출력값)

3. 원칙 · 라이브 데모

검증 게이트 · 무-날조 · provenance · 혹 · 데모 시나리오 · 보안 · 한계

쉽게

이 텍스트는 강사가 Claude Code 화면을 띄워놓고 옆에 두는 '해설서'다. 슬라이드의 모든 수치·출력은 실제로 실행해 확인된 값이다.

프롬프트 한 줄 vs 하네스 — 무엇이 다른가

프롬프트 한 줄에게 시키면

- **모델 기억에 의존**
IC50·ChEMBL ID를 '그럴듯하게' 생성
- **검증 지점 없음**
맞았는지 틀렸는지 확인할 곳이 없음
- **출처 없음**
어디서 온 값인지 되짚을 수 없음
- **재현 불가**
같은 질문에 매번 다른 답
- **실패가 조용함**
모르면 모른다고 하지 않음

하네스에게 시키면

- **도구**
scripts/*.py 가 UniProt·ChEMBL·RDKit 실행
- **계약**
{result, provenance, verification} 고정 스키마
- **게이트**
passed=false → 다음 단계 진행 금지
- **규약**
CLAUDE.md 하드 규칙 + PostToolUse 후
- **정직한 실패**
근거 없으면 '확인 필요' 로 표기

+심화

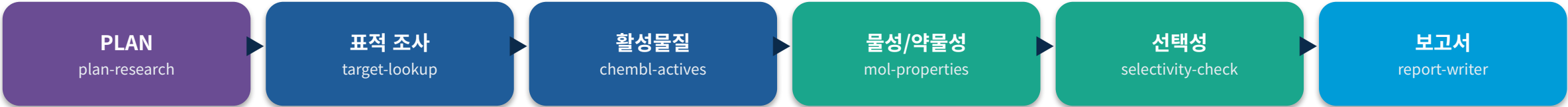
3교시(에이전트 원리)의 '계획 → 도구 → 검증' 루프를, 4교시(하네스 엔지니어링)의 방식으로 실제 폴더에 구현한 것이 이 하네스다.

PART 1

하네스 해부

폴더 구조 · 에이전트 지시서 · 스킬/스크립트 · 출력 계약

폴더 하나가 곧 에이전트 — 구조와 데이터 흐름



↑ 각 단계 사이마다 `verify.py` 게이트 — `passed=false` 면 여기서 멈춘다

pde5_harness/ (실제 구조)

```
pde5_harness/
├── CLAUDE.md           # 에이전트 규약: PLAN→게이트→REPORT, 무-날조
├── README.md  run_harness.py  requirements.txt  Dockerfile
├── .claude/settings.json # PostToolUse 혹은 등록 (matcher: Bash)
├── .claude/hooks/       # verify_provenance.py, no_fabrication_guard.py
├── .claude/skills/<name>/SKILL.md # 스킬 6종
├── scripts/             # verify.py + 단계 스크립트 (표준 봉투 JSON)
└── outputs/             # report_pde5.md, structures/
```

▶ Claude Code 는 이 폴더를 열면 CLAUDE.md(규약)와 .claude/settings.json(혹)을 자동으로 읽는다

CLAUDE.md — 에이전트에게 주는 '실험 프로토콜'

- **목표 고정**
표적 PDE5A (UniProt O76074, ChEMBL CHEMBL1827) · 기전 cGMP 분해 억제 → 혈관확장
- **워크플로 4단계 (순서 엄수)**
PLAN(계획 JSON) → EXECUTE(도구 4단계) → VERIFY GATE(매 단계) → REPORT(IMRAD)
- **하드 규칙**
수치는 도구 실제산값만 · 모든 주장에 출처 · 불확실은 불확실로 · 결과=가설
- **단계별 통과 조건 표**
CLAUDE.md 안에 5개 단계의 합격 기준이 표로 박혀 있다 (뒤에서 상세)
- **재시도 규칙**
passed=false → 최대 2회 재시도 → 그래도 실패면 '미확인/플래그' 로 보고서에 명시
- **스킬·훅 위치 안내**
`.claude/skills/<name>/SKILL.md`, `.claude/hooks/` 를 지시서가 직접 가리킨다

+심화

핵심은 '무엇을 하라'가 아니라 '무엇을 지켜라'. 순서·합격 기준·실패 처리·금지 행위를 문서로 못박아 두면 모델이 바뀌어도 절차가 유지된다.

스킬 6종 — 언제 어떤 도구를 쓸지 알려주는 카드

스킬	역할	호출 스크립트
plan-research	자연어 목표 → 구조화 JSON 계획 (실행 전 필수)	— (JSON 직접 작성)
target-lookup	PDE5A 실재·기능 확인 (UniProt O76074)	scripts/target_lookup.py
chembl-actives	CHEMBL1827 활성물질 조회 (실 ID·SMILES)	scripts/chembl_actives.py
mol-properties	RDKit 물성 · QED · SA · Lipinski 게이트	scripts/mol_properties.py
selectivity-check	PDE5 vs PDE6 선택성 (정직한 한계)	scripts/selectivity.py
report-writer	봉투 종합 → IMRAD 보고서 outputs/report_pde5.md	(verify.py 로 수치 검증)

+심화

SKILL.md 는 frontmatter(name·description) + '언제 쓰는가 / 어떻게 호출하는가 / 반환 검증' 3단 구성. 설명이 곧 트리거 조건이다.

스크립트 7종 — 실제로 계산·조회하는 손발

스크립트	무엇을 하는가	비고
verify.py	make_result / gate / valid_smiles / numbers_backed	공용 검증 유틸
target_lookup.py	UniProt REST 조회 → gene·protein·function·length	오프라인 폴백 있음
chembl_actives.py	chembl_webresource_client 로 활성 레코드 조회	오프라인 폴백 있음
mol_properties.py	RDKit Descriptors/QED/Lipinski/Crippen 실계산	RDKit 없으면 정직 실패
selectivity.py	PDE5/PDE6 아이소자임 맥락 · 교차억제 기전(정성)	정량 예측 안 함
mol_utils.py	RCSB에서 PDB 다운로드 + 경량 텍스트 파싱	옵션
docking.py	외부 smina/vina 바이너리 호출 래퍼	옵션 · 기본은 미실행

**함정**

docking.py 는 자체 물리 엔진이 아니다 — PATH의 smina/vina 를 subprocess 로 부르는 래퍼다. 바이너리·구조·박스가 없으면 실행하지 않고 그렇게 보고한다.

모든 도구가 같은 봉투로 답한다 — {result, provenance, verification}

```
scripts/verify.py :: make_result(result, source, query, checks, notes)
```

```
{
  "result":      { ... },                # 도구가 실제로 계산·조회한 값
  "provenance":  {"source": "UniProt REST",    # 어디서
                  "query":  "https://rest.uniprot.org/uniprotkb/076074.json",
                  "timestamp": "2026-08-31T15:00:19Z"}, # 언제
  "verification": {"passed": true,           # = all(checks)
                  "checks": [{"check": "accession == 076074",
                                "passed": true}, ... ],
                  "notes":  "표적 실재·기능 확인"}
}
```

▶ 계약이 있으면 (1) 다음 도구에 파이프로 넘길 수 있고 (2) 후이 기계적으로 검사할 수 있고 (3) 보고서에서 역추적된다

PART 2

도구 5종 — 실제 출력

실행해서 확인된 값만 인용한다

target_lookup.py — 표적이 실재하는가 (UniProt)

```
$ python scripts/target_lookup.py
```

```
"result": {"accession": "076074", "gene": "PDE5A",  
          "protein": "cGMP-specific 3',5'-cyclic phosphodiesterase",  
          "function": "... catalyzes the specific hydrolysis of cGMP to  
                      5'-GMP (PubMed:15489334, PubMed:9714779) ...",  
          "length": 875},  
"provenance": {"source": "UniProt REST",  
              "query": "https://rest.uniprot.org/uniprotkb/076074.json"},  
"verification": {"passed": true, "checks": [  
    accession == 076074 / phosphodiesterase 5 언급 / 기능 서술 존재 ]}
```

▶ 데모 포인트: 하드코딩이 아니라 방금 REST 를 때린 응답. query 필드에 URL 이 그대로 남는다

+심화

네트워크가 막히면 공개 참조값으로 폴백하되 source 에 'UniProt (OFFLINE DEMO)' 를 붙인다 — 값을 만들지 않고, 값의 출처 등급을 낮춰 표시한다.

chembl_actives.py — 후보 화합물 (실 ID · 실 SMILES)

```
$ python scripts/chembl_actives.py 3
```

```
"result": [  
  {"molecule_chembl_id": "CHEMBL192", "pref_name": "SILDENAFIL",  
   "canonical_smiles": "CCCC1nn(C)c2c1nc([nH]c2=O)-c1cc(ccc1OCC)S(=O)(=O)N1CCN(C)CC1"},  
  {"molecule_chembl_id": "CHEMBL779", "pref_name": "TADALAFIL", ...},  
  {"molecule_chembl_id": "CHEMBL1520", "pref_name": "VARDENAFIL", ...} ],  
"provenance": {"query": "target_chembl_id=CHEMBL1827; type in (IC50,Ki)"},  
"verification": {"passed": true, "checks": [  
  레코드 존재 / 모든 SMILES RDKit 유효 / 모든 ID 가 ChEMBL 접두 } ]}
```

▶ 데모 포인트: SMILES 는 RDKit 로 파싱되는 것만 통과 — 파싱 안 되면 '가짜'로 간주하고 게이트가 막는다



합정

chembl_webresource_client 미설치 환경에서는 오프라인 폴백이 돌고, source 에 'ChEMBL (OFFLINE DEMO — 실데이터 아님)' 이 찍힌다. ID · SMILES 는 실재값이지만 라이브 조회는 아니다.

mol_properties.py — RDKit 실계산 물성 · 약물성 게이트

```
$ python scripts/mol_properties.py "<sildenafil SMILES>"
```

```
"result": [{ "MW": 474.6, "logP": 1.61, "HBD": 1, "HBA": 7,
             "TPSA": 113.4, "QED": 0.553, "SA": 2.74,
             "Ro5_pass": true, "gate_pass": true }],
"provenance": {"source": "RDKit Descriptors/QED"},
"verification": {"passed": true,
                 "checks": [후보 존재 / 값 sanity 범위 내],
                 "notes": "게이트(Ro5≥3 · QED≥0.5 · SA≤6.0) 통과 1건"}
```

```
# 게이트: Ro5 4항목 중 3개 이상 충족 AND QED ≥ 0.5 AND (SA 있으면) SA ≤ 6.0
```

▶ 데모 포인트: 이 7개 숫자는 LLM 이 쓴 게 아니라 RDKit 이 SMILES 로부터 계산한 값이다



합정

QED≥0.5 · SA≤6.0 은 코드에 박힌 교육용 데모 임계값(QED_MIN, SA_MAX)이다. 실제 과제에서는 목적에 맞게 바꿔야 한다. SA 는 RDKit contrib(sascorer) 없으면 null 로 남긴다.

selectivity.py — 못 하는 것을 못 한다고 말하는 도구

```
$ python scripts/selectivity.py
```

```
"result": {"quantitative_selectivity_predicted": false,
  "assessment": "정성(qualitative) — 정량 fold-selectivity 미산출",
  "isoform_context": {PDE5: 혈관 평활근 ..., PDE6: 망막 광수용체 ...},
  "cross_inhibition_mechanism": "망막 PDE6 교차 억제 → 일시적
    청녹 색각 이상(cyanopsia) ...",
  "confirmation_required": [병렬 IC50/Ki 어세이, ChEMBL 병렬 조회,
    문헌 선택성 비의 PMID/DOI 명시]},
"verification": {"passed": true}    # ← 수치를 안 낸 것이 '통과' 조건
```

▶ 데모 포인트: 첫 번째 check 항목이 '정량 fold-selectivity 미날조(정성 보고)' 다 — 억지 수치를 내면 오히려 실패

+심화

참고문헌도 실재하는 2편만 반환한다 (Booell 1996 Int J Impot Res; Ghofrani 2006 Nat Rev Drug Discov, doi:10.1038/nrd2030).

verify.py — 게이트 그 자체 (모든 도구가 import 한다)

scripts/verify.py — 4개 함수가 전부다

```
make_result(result, source, query, checks, notes)    # 표준 봉투 생성
    passed = all(ok for _, ok in checks)             # 하나라도 False → 실패

gate(envelope, step) -> bool                         # 진행/중단 판정
    stderr: "[VERIFY-GATE:target-lookup] PASSED"      # 화면에 찍힌다
    stderr: "[VERIFY-GATE:...] FAILED → 다음 단계 진행 금지"

valid_smiles(smiles) -> bool                         # RDKit 파싱 가능한가
numbers_backed(text, allowed_numbers) -> list        # 근거 없는 수치 목록
```

▶ report-writer 는 보고서 본문의 숫자를 numbers_backed 로 훑어 앞 단계 결과에 없는 수치를 잡아낸다

⚠ **함정**

numbers_backed 는 정규식으로 숫자 문자열을 뽑아 비교하는 휴리스틱이다 — 완벽한 사실 검증기가 아니라 '눈에 띄는 날조'를 잡는 1차 그물이다.

mol_utils.py / docking.py — 있으면 쓰고, 없으면 정직하게 스킵

mol_utils.py (구조)

- **RCSB에서 PDB 파일 다운로드**
files.rcsb.org/download/<code>.pdb
- **의존성 없는 경량 텍스트 파서**
RDKit · MDAnalysis 불필요
- **파싱 결과**
해상도 · 원자수 · 체인 · HETATM 리간드
- **PDE5A 참조 코드 5종**
1UDT · 1XP0 · 1XOZ · 1UDU · 2H42
- **실패 시**
파일을 만들지 않고 downloaded:false

docking.py (도킹)

- **자체 도킹 엔진 아님**
PATH의 smina/vina 를 subprocess 호출
- **전제 4가지 점검**
바이너리 · 수용체 · 리간드 · 박스(center/size)
- **하나라도 없으면**
docking_executed:false + missing 목록
- **스코어**
stdout 파싱값만, 없으면 null
- **실행돼도 caveat**
도킹 스코어는 가설 — 친화도 실측 아님

+심화

기본 데모에서 docking.py 는 '미실행'을 반환한다. 이것이 버그가 아니라 설계다 — 실행 못 한 계산의 결과를 지어내지 않는다.

PART 3

원칙 · 라이브 데모

게이트 · 무-날조 · provenance · 혹 · 시나리오 · 보안 · 한계

단계별 검증 게이트 — CLAUDE.md 에 박힌 통과 조건

단계	통과 조건 (CLAUDE.md 원문)
target-lookup	반환 텍스트에 phosphodiesterase 5 / PDE5A 포함 + accession = O76074
chembl-actives	각 레코드에 실제 molecule_chembl_id + SMILES 가 RDKit 로 파싱 + 단위 정상
mol-properties	MW/logP/HBD/HBA/TPSA/QED 가 물리적 범위 내 ($0 < MW < 2000$, $-10 < \log P < 15$, $0 \leq QED \leq 1$)
selectivity-check	정량 예측이 없으면 '정성 · 확인 필요' 로 명시 (억지 수치 금지)
report	모든 수치가 앞 단계 도구 결과에 존재 (근거 없는 수치 0) + 참고문헌 실재

+심화

게이트가 없으면 에이전트는 '대충 그럴듯한 다음 단계'로 미끄러진다. 통과 조건을 문서 + 코드 양쪽에 두면 미끄러질 자리가 없어진다.

실패를 실패라고 말하는 설계 — 실제 출력

후보가 0건일 때 (SMILES 인자 없이 실행)

```
$ python scripts/mol_properties.py          # 입력 SMILES 없음
"result": [],
"verification": {"passed": false,
  "checks": [{"후보 존재": false}, {"값 sanity 범위 내": false}],
  "notes": "물성 계산 0건, 게이트 통과 0건"}

# 여기서 파이프라인은 멈춘다. 빈 표도, 평균값도, 대표값도 만들지 않는다.
```

▶ RDKit 미설치 시에도 동일 — 수치 대신 {'RDKit 설치': false} 와 'pip install rdkit' 안내만 반환한다

+심화

selectivity.py 가 정량 fold-selectivity 를 '안 내는' 것도 같은 원칙이다. 모르는 것을 모른다고 하는 것이 이 하네스의 유일한 성공 조건에 가깝다.

출처 추적 — 모든 값에 source · query · timestamp

- **source — 어디서 왔는가**
'UniProt REST' / 'RDKit Descriptors/QED' / 'ChEMBL (OFFLINE DEMO — 실데이터 아님)'
- **query — 무엇을 물었는가**
URL 전문 또는 'target_chembl_id=ChEMBL1827; type in (IC50,Ki); n=3'
- **timestamp — 언제 물었는가**
UTC ISO8601 — 같은 DB라도 시점이 다르면 값이 다를 수 있다
- **등급 표시**
라이브 조회인지 오프라인 폴백인지가 source 문자열로 구분된다
- **보고서까지 전달**
report-writer는 각 사실 주장에 ChEMBL ID / UniProt / PMID / DOI 를 붙인다

+심화

2교시 환각 통제의 실물판: '모델이 뭐라고 했는가'가 아니라 '어느 API가 언제 뭐라고 답했는가'를 기록한다. 출처가 없으면 그 문장은 '확인 필요'로 내려간다.

PostToolUse 후 2종 — Bash 실행마다 자동으로 끼어든다

verify_provenance.py

- **이벤트**
PostToolUse · matcher: Bash
- **점검**
출력에 provenance / verification 필드가 있는가
- **passed:false 감지 시**
'다음 단계 전 재시도/플래그 검토' 경고 추가
- **주입 방식**
hookSpecificOutput.additionalContext

no_fabrication_guard.py

- **점검 1**
의심 표현: 추정 · 대략 · estimated · plausible ...
- **점검 2**
형식이 깨진 식별자 (ChEMBL 뒤 숫자 없음)
- **걸리면**
'도구 실제값만 인용' 리마인더 주입
- **안 걸려도**
무-날조 원칙 리마인더를 매번 남김



함정

두 후 모두 경고형(비차단)이다 — exit 0 으로 끝나며 도구 실행을 막지 않는다. 데모 가시성을 위한 설계이고, 실제 차단은 verify.py 의 gate() 가 담당한다.

강사가 실제로 칠 순서

터미널 → Claude Code

```
$ cd 0905_agent/pde5_harness
$ python run_harness.py check          # ① 환경 점검
[dep] rdkit ok / [script] 5개 ok / [rdkit] ethanol MW=46.07 ok
결과: PASS — 자율 실행 준비 완료

$ python run_harness.py list           # ② 스킬·스크립트 목록 보여주기
$ python scripts/target_lookup.py      # ③ 도구 하나 맨손 실행 (봉투 보여주기)

$ claude                              # ④ 여기서부터 에이전트가 운전
> "CLAUDE.md 읽고 PDE5 저해제 후보 조사해 보고서까지 자율 실행해줘"
```

▶ run_harness.py run 을 쓰면 에이전트 없이 target→chembl→properties→selectivity 를 파이썬이 순차 실행한다 (게이트 포함)

수강생이 화면에서 봐야 할 다섯 가지

- ① 계획을 먼저 쓴다
도구를 건드리기 전에 objective/questions/tools/success_criteria JSON 부터
- ② 도구 선택의 근거
왜 지금 chembl-actives 인가 — SKILL.md 의 '언제 사용하는가'가 트리거
- ③ 게이트 로그
stderr 의 [VERIFY-GATE:<step>] PASSED / FAILED 한 줄
- ④ 실패 처리
0건·네트워크 실패 시 재시도 → 폴백 → '미확인 플래그'. 수치를 만들지 않는지
- ⑤ 혹 리마인더
Bash 실행마다 [verify_provenance] / [no_fabrication_guard] 문구가 붙는지

쉽게

강사 팁: ③④를 보여주려면 네트워크를 끊거나 잘못된 accession(예: O00000)을 일부러 주고 에이전트가 어떻게 회복하는지 관찰시키면 좋다.

이 하네스에 실제로 있는 것 / 실무에서 더 붙여야 할 것

하네스에 실제로 구현된 것

- **API 키 불필요**
UniProt · ChEMBL · RCSB 공개 REST 만 사용
- **컨테이너 격리**
Dockerfile · outputs/ 만 VOLUME 으로 노출
- **자동 감사 훅**
PostToolUse 2종이 매 Bash 호출을 점검
- **graceful 폴백**
네트워크 · 라이브러리 실패에 죽지 않음
- **쓰기 범위 제한**
산출물은 outputs/ 로 한정

실무 배포 시 추가 (4교시)

- **키 관리**
환경변수 · 시크릿 매니저, 코드 · 로그에 금지
- **권한 최소화**
도구별 허용 명령 화이트리스트
- **차단형 훅**
경고형 → PreToolUse 차단형으로 승격
- **감사 로그 보존**
호출 · 응답 · 게이트 판정 영속화
- **네트워크 통제**
허용 도메인 외 egress 차단

+심화

에이전트 보안의 출발점은 '무엇을 실행할 수 있는가'의 축소다. 이 하네스는 공개 DB 읽기 + 로컬 계산만 하므로 공격면이 작다 — 사내 데이터가 붙는 순간 이야기가 달라진다.

정직하게 — 이 하네스가 하지 않는 것

- **도킹은 실제 물리 도킹이 아니다**
docking.py 는 외부 smina/vina 래퍼. 기본 데모에서는 아예 미실행
- **정량 선택성 없음**
PDE5 vs PDE6 는 정성 서술만 — fold-selectivity 는 병렬 어세이·문헌 필요
- **오프라인 풀백은 실데이터가 아니다**
대표 3개 화합물의 공개 ID·SMILES 일 뿐, 활성값 없음
- **혹은 차단하지 않는다**
경고형 · 리마인더 주입만. 실제 차단은 verify.py gate()
- **임계값은 교육용**
 $QED \geq 0.5$ · $SA \leq 6.0$ 은 데모 상수 (QED_MIN, SA_MAX)
- **결과 = 가설**
알려진 화학공간의 선별·정리 시연이며 신약 발견이 아니다



함정

LLM 은 여전히 환각한다. 이 하네스가 줄이는 것은 '수치·식별자 날조'이지 '해석의 오류'가 아니다 — 보고서의 논증은 사람이 읽어야 한다.

Claude Code 구독이 없다면 — nb1 노트북으로 같은 개념

nb1_scientific_agent.ipynb (무료)

- **Colab · 완전 무료**
rdkit + requests 만으로 무-키 실행
- **선택: Gemini 무료 티어**
GOOGLE_API_KEY 넣으면 진짜 LLM 이 도구 선택
- **도구 5종 (전부 실 API·실계산)**
search_pubmed · lookup_uniprot · search_chembl_actives ·
mol_properties · admet_flags
- **에이전트 루프를 손으로 구현**
call_tool · audited_call · with_retry · run_agent

pde5_harness (강사 데모)

- **실행 주체**
Claude Code 에이전트가 파일·Bash·혹을 직접
- **계획**
CLAUDE.md 규약 기반 자율 PLAN
- **검증**
PostToolUse 혹 + verify.py 단계 게이트
- **도구**
SKILL.md 6종 + scripts 7종
- **비용**
Claude Code 유료 구독 필요

+심화

배우는 개념은 동일하다 — 도구 등록 · 표준 반환 · 재시도 · 출처 기록 · 검증. 노트북은 그 루프를 파이썬으로 직접 쓰고, 하네스는 에이전트에게 맡긴다.

이 데모에서 가져갈 다섯 가지

1

하네스 = 도구 + 계약 + 게이트 + 규약

프롬프트가 아니라 폴더 구조가 에이전트의 행동을 결정한다

2

표준 봉투 {result, provenance, verification}

체인닝·자동검사·역추적이 전부 이 계약 하나에서 나온다

3

게이트는 문서와 코드 양쪽에

CLAUDE.md 통과 조건 표 + verify.py gate() — 미끄러질 자리를 없앤다

4

무-날조는 기능이다

후보 0건 → passed=false · 정량 선택성 미산출 — 모르면 모른다고 한다

5

provenance 가 신뢰의 단위

source·query·timestamp 가 없으면 그 문장은 '확인 필요'

다음

같은 뼈대, 다른 표적 — 항체 설계 하네스

antibody_harness/ — antigen_lookup → antibody_search → cdr_analysis
→ developability → humanness (기본 표적 HER2, UniProt P04626)

소분자에서 바이오희스로 바뀌어도 계약·게이트·무-날조 구조는 그대로다.

먼저 해볼 것: run_harness.py check → 자연어 한 줄 → 화면을 지켜보기.